Make Graphs Great Again: Rethinking LLM Agent Architecture via Structure–Semantic Separation

Jingyi Jia
Huazhong University of Science and Technology
Wuhan, China
jingyijia@hust.edu.cn

Jiale Wang
Huazhong University of Science and Technology
Wuhan, China
chiallllee@hust.edu.cn

Haoyi Zhang
Huazhong University of Science and Technology
Wuhan, China
haoyizhang@hust.edu.cn

Qinbin Li*
Huazhong University of Science and Technology
Wuhan, China
qinbin@hust.edu.cn

Xiaofei Liao
Huazhong University of Science and Technology
Wuhan, China
xfliao@hust.edu.cn

Hai Jin
Huazhong University of Science and Technology
Wuhan, China
hjin@hust.edu.cn

Abstract

LLM-centric agent architectures route each observation back to the model for the next decision, leaving control logic implicit in a growing prompt and forcing repeated reconstruction of task state and routine execution progress even when runtime evidence determines the next transition.

We present **structure–semantic separation** as a control boundary for LLM agents, making recurring tool-use structure explicit and executable while reserving the model for semantic decisions that structure cannot resolve. MotifAgent realizes this boundary through executable motif graphs: the Structure Engine advances graph execution based on runtime evidence, while the Semantic Engine resolves semantic gaps through scoped LLM calls, making recurring tool-use structure executable runtime state rather than prompt guidance. Exposing motif progress to the serving backend additionally enables structure-aware scheduling and fine-grained KV-cache retention. Across APIBank, TauBench, and AppWorld, MotifAgent reduces LLM calls and token consumption while maintaining comparable task performance. Serving replay experiments further show reduced prefill-stage latency and improved admission-time KV reuse under the evaluated queueing and cache-pressure conditions.

*Qinbin Li is the corresponding author. All authors are with the National Engineering Research Center for Big Data Technology and System, Service Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, Hubei, China.



This work is licensed under a Creative Commons Attribution 4.0 International License.

SIGOPS ATC 2026, Hong Kong, China
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2818-1/2026/11
https://doi.org/10.1145/3828523.3857104

CCS Concepts: • **Computing methodologies** → **Artificial intelligence**.

Keywords: LLM agents, agent runtime, LLM serving, KV cache

ACM Reference Format:
Jingyi Jia, Jiale Wang, Haoyi Zhang, Qinbin Li, Xiaofei Liao, and Hai Jin. 2026. Make Graphs Great Again: Rethinking LLM Agent Architecture via Structure–Semantic Separation. In *The 2026 ACM SIGOPS Annual Technical Conference (SIGOPS ATC 2026), November 15-18, 2026, Hong Kong, China.* ACM, New York, NY, USA, 16 pages. https://doi.org/10.1145/3828523.3857104

1 Introduction

Enabled by the powerful abilities of Large Language Models (LLMs), LLM agents [30, 43, 46, 49, 52] are increasingly used in various domains such as coding, web automation, and personal assistance. Because LLMs can flexibly interpret new observations and decide subsequent actions in natural language, an LLM-centric agent architecture has become a natural default: after each observation, control is returned to the LLM, which decides the next step. This design preserves the generality and flexibility of LLMs, but it also makes the core control logic of the agent program highly dependent on LLM calls.

This convenience is not free. Each decision step introduces an expensive LLM call with growing context. The high cost of this architecture has motivated many efforts to accelerate LLM agents, such as batching, scheduling, and KV-cache management [17, 53, 59]. These techniques are effective, but most still optimize within the LLM-centric loop. They make model-mediated steps cheaper and faster, while the agent often still returns to the LLM after each observation, even when much of the execution is already structurally determined. As a result, such progress remains coupled to repeated LLM inference over growing context.

We argue that this bottleneck is architectural rather than merely computational. In many agent executions,

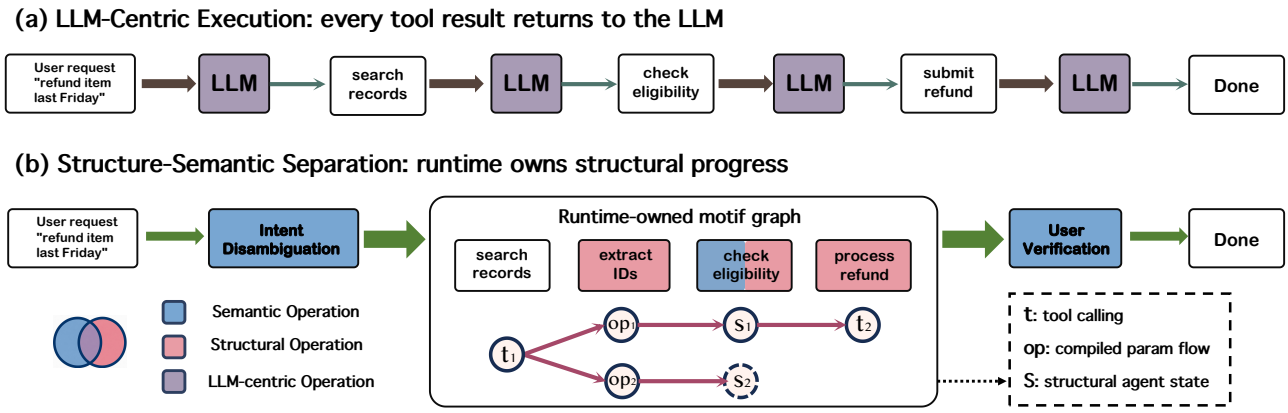


Figure 1. Running example of a refund task. (a) LLM-centric execution returns control to the LLM after every tool result. (b) Structure–semantic separation leaves semantic decisions to the LLM while executing stable tool order and parameter flow as a motif graph. Mixed colors denote evidence-dependent execution modes.

especially after the initial ambiguity has been resolved, the next step is not always an open-ended semantic decision. It may follow a stable tool order, transfer a value from one tool result to a later input, release a downstream operation after its dependency is satisfied, or check a known precondition. Under an LLM-centric architecture, these structural transitions remain encoded in prompt text rather than executable runtime state. The runtime observes tool calls and tool results, but it does not own the recurring structure that connects them.

Tool use makes this opportunity especially clear. Tool calls expose explicit runtime boundaries, including input schemas, returned fields, and parameter dependencies across calls. These signals make the execution structure observable to the runtime, unlike consecutive model-only reasoning steps whose boundaries depend heavily on prompting choices and model behavior. Different tasks may also share local tool-use structure even when their end-to-end workflows differ.

The missing piece is a **redesign of control ownership**: the runtime should advance structural progress when current evidence is sufficient, while the LLM should be reserved for semantic gaps that structure cannot resolve.

Figure 1 illustrates this boundary with a refund task. In LLM-centric execution (a), the agent returns to the model after each tool result to choose the next step. Under structure–semantic separation (b), "the item I bought last Friday" still requires semantic mediation because the target is ambiguous. Once the relevant record is identified, the runtime can follow a motif graph to extract the order identifier, check eligibility, and process the refund accordingly. Each transition proceeds only when the required values and preconditions are verified.

If required evidence is missing, the runtime invokes scoped semantic mediation to resolve the gap. The model receives only the current motif state and the evidence needed to resolve that gap, rather than the full execution history. This shows that the boundary between structural execution and semantic reasoning is determined dynamically at runtime: verifiable transitions are executed structurally, while unresolved ones are delegated to the LLM. We call this boundary structure–semantic separation.

We build MotifAgent, an **agent-runtime and serving co-design system** that realizes structure–semantic separation. It represents recurring tool-use structure as motif graphs. *Positive motifs* encode local tool operations, parameter flow, dependencies, and the evidence required for each transition. These motifs can be reused across tasks and connected as execution progresses. The Structure Engine instantiates a matched motif as runtime state and advances its execution only when the required values and preconditions are verified. At a semantic gap, the Semantic Engine receives a typed mediation request and invokes the LLM for bounded decisions such as value completion, candidate selection, or clarification. *Negative motifs* are extracted from failed trajectories. They support early stopping on known failure patterns and provide corrective guidance during ReAct fallback.

Runtime-owned motifs also change what the serving system can observe. A conventional backend sees arrived LLM calls and token-level features, but not the dynamic local tool workflow or downstream operations behind each call. MotifAgent exposes motif stage, structured prompt prefix, and possible downstream releases. These serving-visible signals guide structure-aware scheduling and fine-grained KV-cache retention and eviction.

We evaluate structure–semantic separation on APIBank [20], TauBench [51], and AppWorld [40]. Compared with LLM-centric execution, MotifAgent reduces LLM calls by up to 27.1% and token consumption by up to 47.9%, while maintaining competitive task performance across the three benchmarks. The serving backend also uses motif progress to guide

scheduling and retain prefixes likely to be reused after tool gaps.

Our contributions are summarized as follows:

- We identify the LLM-centric agent design as an architectural bottleneck and propose structure–semantic separation as a new execution boundary. It moves verifiable structural progress to the runtime and reserves semantic judgment for the LLM.
- We build MotifAgent, an agent-runtime and serving co-design system. Its motif graphs capture recurring local tool-use structure across tasks, advance evidence-supported transitions, and scope LLM calls at semantic gaps.
- We design a serving interface and backend mechanisms that expose motif stages, structured prompt prefixes, and downstream-release hints. These signals guide structure-aware scheduling and fine-grained KV-cache retention and eviction.
- We implement MotifAgent on an SGLang-based serving stack. Across three agent benchmarks, it reduces LLM calls and token use at comparable task performance. Controlled replays evaluate latency and KV-cache effects under pressure.

2 Background

2.1 LLM Agent Control Loops

LLM agents interleave model decisions with observations from users, tools, and the environment [30, 43, 46, 52]. In a ReAct-style loop [52], the model selects an action, observes the result, and then decides the next step. This lets the agent respond to unexpected observations without a predefined workflow. Tool calls expose explicit runtime structure, such as typed inputs and outputs and parameter dependencies across operations. However, the recurring control logic connecting these calls often remains implicit in the growing prompt history. When the same pattern recurs, the model is asked to interpret it again instead of the runtime executing it as state.

Several agent and language-model programming systems try to reduce the cost of repeated model calls through tool-use training, reusable prompts, declarative pipelines, structured generation, or workflow-style execution [4, 9, 15, 28, 37, 44, 58, 59]. Agent Skills capture reusable procedural knowledge through instructions, scripts, and resources, using progressive disclosure to load relevant information only when needed [54]. These systems improve knowledge reuse and the organization of LLM calls, but deciding when and how to apply reusable knowledge often remains model-mediated. In this paper, we focus on a complementary boundary: when execution progress is already verifiable from runtime evidence, the runtime should advance it directly rather than asking the LLM to rediscover the same control structure.

2.2 LLM Serving and Prefix Caching

LLM serving systems batch requests to improve accelerator utilization. For decoder-only models, prefill processes the prompt and constructs KV states, while decode generates output tokens autoregressively. Serving systems such as ORCA and SARATHI improve utilization through iteration-level scheduling and chunked prefills [2, 53], while vLLM and others improve memory efficiency through PagedAttention and block-level KV-cache management [17, 27, 33]. Other recent architectures further improve serving efficiency by disaggregating prefill and decode [26, 60].

Prefix reuse is especially important for multi-call LLM applications [9, 34, 50]. Caching KV states for shared prompt prefixes avoids recomputing the shared part of prefill. SGLang [59] targets structured language-model programs and introduces RadixAttention, which stores reusable prefixes in a radix tree and reuses cached KV states across calls. Its scheduler can use radix-aware policies to select waiting requests based on token-prefix locality. Longest-prefix matching favors requests whose token prefix is already well represented in the cache, while DFS-style traversal favors dense prefix subtrees to improve locality.

However, agent requests arrive sequentially, separated by tool execution delays known as tool gaps [1, 19]. An LRU-based cache can evict a prompt prefix during a long tool gap, causing a cache miss when the tool returns.

Furthermore, these policies operate on tokenized prompt prefixes. Recent application-aware systems expose request dependencies or workflow structure to the serving layer [22, 42]. In LLM agents, tool observations, semantic gaps, and fallbacks dynamically change the execution state and which downstream operations become ready.

Token-prefix information and request dependencies alone do not fully describe an agent's evolving execution state. Exposing this state would help the backend anticipate downstream operations and future prefix reuse across tool gaps. This information gap between the agent runtime and serving backend motivates our cross-layer co-design.

3 Motivation

Structure–semantic separation shifts verifiable tool-use steps to the runtime and exposes execution state to the serving backend. The runtime reuses recurring local tool-use structure when current evidence supports its transitions (§3.1). It delegates unresolved decisions to the LLM through bounded semantic requests (§3.2). The backend uses the exposed progress to guide scheduling and cache retention (§3.3).

3.1 Observation 1: Local Tool-Use Structure Recurs Across Tasks

ReAct-style agents leave execution control entirely to the LLM for each task. However, tool-use traces from different tasks often share local operation sequences, parameter flows,

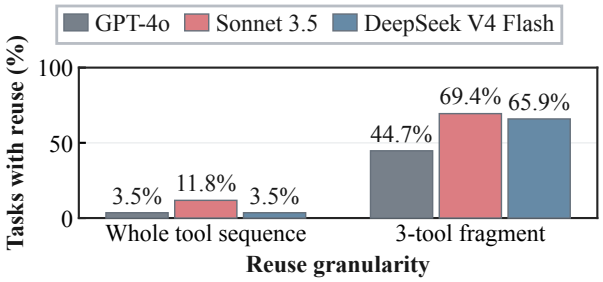


Figure 2. Cross-task recurrence on TauBench Retail. Each model uses one trajectory for each of the same 85 tasks.

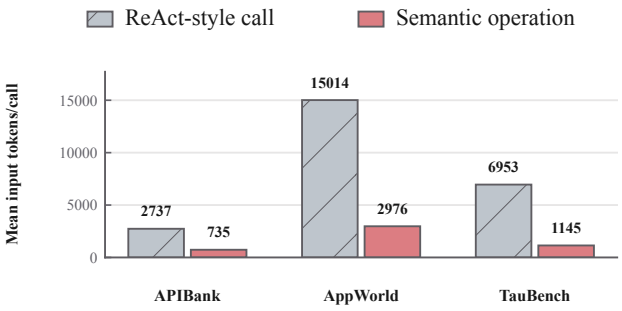


Figure 3. Semantic operations reduce input tokens by using compact motif state and local evidence.

and dependency checks, even when their complete execution paths differ. The LLM repeatedly reasons over these recurring structures, leading to additional model calls and token costs.

For example, a refund request and a shipment inquiry may both retrieve an order and extract its identifier before taking different downstream actions. Their complete workflows differ, but the shared fragment follows the same tool order and parameter dependencies. This local repetition creates reuse opportunities beyond whole-task workflow matching.

As Figure 2 shows, 44.7–69.4% of tasks share a three-tool fragment with another task across three models on TauBench Retail [51]. In contrast, only 3.5–11.8% share their complete tool sequence.

Motifs capture these fragments as executable graphs mined from historical traces. Runtime evidence determines when a matched fragment can execute and helps identify its possible successors. As execution progresses, new evidence can enable motif re-matching or continuation through connected motifs. The runtime can therefore reuse local structure without prescribing the entire task path.

Challenge 1: Reuse local structure as evidence evolves. The runtime must retrieve candidates, ground arguments in current evidence, and verify preconditions and dependencies before advancing. It must also support partial execution, continuation across motifs, and re-entry after unresolved decisions. This requires retaining validated progress while updating execution state and successor choices as new evidence arrives.

3.2 Observation 2: Structure Cannot Replace Semantics, But Can Bound It

Structure alone does not resolve every decision in agent execution. User requests may omit required values, refer to entities indirectly, or match multiple candidate motifs. Tool results may fail to satisfy expected guards. In these cases, the runtime may lack the evidence needed to determine tool arguments or validate the next transition. The question is therefore where to place the boundary between runtime execution and semantic reasoning.

MotifAgent draws this boundary through two runtime operations. A *structural operation* runs when current evidence is sufficient: it may invoke a known tool node, copy a verified value, check a guard, or release a downstream dependency. When evidence is insufficient, the runtime initiates a *semantic operation* to resolve a typed local gap, such as a missing slot, ambiguous entity, unexpected observation, or failed guard. During semantic mediation, the LLM answers a bounded question using motif state and compact evidence, while the runtime retains execution state.

Figure 3 compares input tokens per call across APIBank [20], TauBench [51], and AppWorld [40]. Semantic operations use fewer input tokens than ReAct-style calls by supplying compact motif state and relevant local evidence instead of the full execution history.

Challenge 2: Bound semantic mediation while preserving runtime state. When structure is insufficient, the LLM must address the local gap while the runtime retains execution state. A typed mediation request specifies the unresolved decision and its relevant evidence. The LLM provides a response, which the runtime returns to the paused motif instance.

3.3 Observation 3: Explicit Runtime State Exposes Serving Opportunities

Structure–semantic separation not only reduces model calls but also makes execution progress and dependencies explicit to the runtime. By tracking completed operations and unresolved dependencies, the runtime can anticipate potential downstream semantic calls within each motif. Such information is not directly visible to a serving backend that only observes submitted requests.

This visibility creates opportunities for scheduling and prefix-cache retention. We leverage motif progress to reduce queueing delays through scheduling and avoid redundant prefill through KV-cache retention across tool gaps.

Realizing these opportunities requires runtime–backend co-design. The runtime exposes motif progress and potential downstream calls, while the backend combines these hints

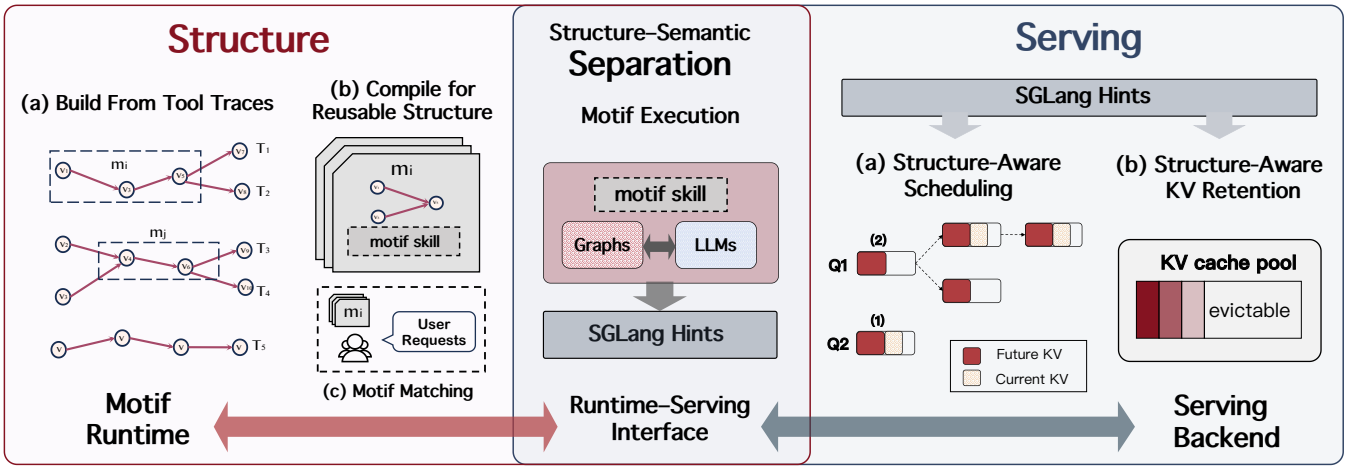


Figure 4. MotifAgent's system architecture

with its queue, running batch, and radix-cache state to guide scheduling and cache retention.

Challenge 3: Balance downstream reuse against current locality and cache pressure. Prioritizing potential downstream reuse may reduce current prefix locality and increase waiting time. Retaining prefixes for uncertain future requests may displace other useful cached context. The challenge is to balance these costs against expected downstream benefits.

These observations motivate three design requirements: evidence-validated local structural reuse, bounded semantic mediation, and serving policies that balance downstream reuse with current locality and cache pressure. The next section describes how MotifAgent realizes these requirements.

4 MotifAgent Design

The key design choice in MotifAgent is to make recurring tool-use structure executable runtime state rather than textual guidance for the LLM. Motifs capture reusable local operations and parameter flow. Runtime evidence determines which transitions can proceed without the model, while explicit motif progress also informs serving decisions.

4.1 Overview

Figure 4 shows the agent runtime, system interface, and serving backend. An offline compiler builds motifs from historical tool-use trajectories. At task time, a matched positive motif becomes an execution instance that records completed operations, tool observations, argument bindings, and unresolved requirements. Unlike a fixed workflow, it represents a reusable fragment rather than an entire task path.

The Structure Engine advances the fragment when current evidence satisfies its arguments and guards. Only unresolved semantic decisions are delegated to the Semantic Engine through typed, scoped requests. Tool results and semantic responses update the same runtime-owned instance, allowing execution to continue without reconstructing prior progress in the LLM. Each motif stage identifies a discrete point in a motif's execution. The runtime exposes stage identifiers and reusable prefixes through `sglang_hints` for backend scheduling and KV-cache retention.

Execution bound. Motifs can bind arguments, propagate values, and execute verified tool transitions without LLM calls. Yet some decisions, such as interpreting ambiguous intent or performing complex reasoning, cannot be resolved from runtime evidence and structural rules alone. These necessary semantic decisions separate stretches of model-free execution. When a later decision depends on tool evidence produced after an earlier one, the two cannot be resolved in the same model invocation.

For a fixed execution path of task τ with at least one required model-resolved decision, let $\mathcal{D}_\tau = (V, E)$ be its semantic dependency DAG. Each node represents a required semantic decision, with repeated occurrences represented by distinct nodes. An edge $(u, v) \in E$ means that v requires evidence available only after u and unavailable within the same model invocation. The *semantic critical path* counts decision nodes on the longest directed path:

$$\mathrm{SCP}(\tau) = \max_{\pi \in \mathrm{Paths}(\mathcal{D}_\tau)} |\pi| \tag{1}$$

Independent decisions may share an LLM round, but decisions along this path must remain sequential. They require at least $\mathrm{SCP}(\tau)$ LLM rounds. If a step-by-step baseline follows the same path with $N_{\mathrm{step}}(\tau)$ model-mediated rounds, its maximum reduction factor is bounded by

$$\mathrm{RoundReduction}_{\max}(\tau) \le \frac{N_{\mathrm{step}}(\tau)}{\mathrm{SCP}(\tau)} \tag{2}$$

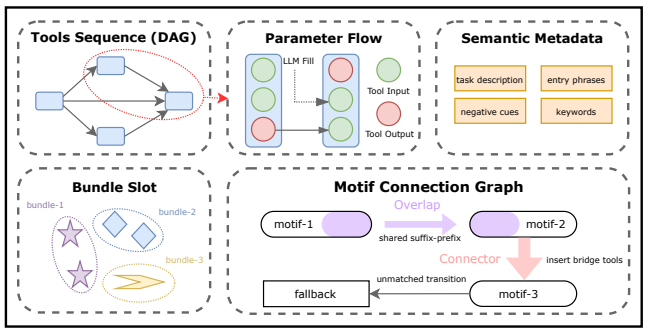


Figure 5. An executable motif combines an operation graph and parameter-flow rules with metadata for semantic resolution and continuation.

Motifs remove rounds for covered structural transitions. Unresolved semantic decisions and fallback determine how closely execution approaches this bound.

4.2 Executable Motifs and Adaptive Composition

The unit of reuse in MotifAgent is a local executable fragment, not an end-to-end workflow or a prompt instruction. *Positive motifs* encode recurring tool-use structure from successful trajectories. *Negative motifs* capture recurring failure patterns and can guide recovery or stop repeated failures during ReAct fallback. Both are compiled offline, while the runtime determines their applicability from current evidence.

Motif representation. A motif reuses the rules for obtaining arguments, not the values observed in a previous task. As shown in Figure 5, its operation graph captures tool calls, dependencies, and parameter flow. A rule can copy a prior tool output, extract a verified value from local evidence, or request semantic resolution when evidence is insufficient. Thus, the same fragment can execute with different task-specific values without asking the LLM to reconstruct its tool transitions.

The graph also supports bounded LLM filling, including batched resolution of missing arguments, response generation, and mediation handoff. Connection hints record shared boundaries and observed bridging tools between fragments. They support dynamic continuation and help identify possible downstream calls without fixing the full task path in advance.

Offline compilation. The compiler discovers recurrent structure from traces, then checks whether that structure is supported by tool schemas and execution evidence. Algorithm 1 normalizes trajectories and mines recurring contiguous fragments from successful executions. It builds operation graphs, attaches support evidence for parameter-flow rules, and records connections between fragments.

Tool order alone cannot establish every parameter rule or semantic requirement. The LLM therefore proposes parameter-flow rules, tool effects, and parallelism annotations from each

Algorithm 1 Motif compilation

Require: Trajectories $\mathcal{T}$, tool schemas Σ
Ensure: Positive motifs $\mathcal{P}$, negative guards $\mathcal{N}$, connector table C

$\widehat{\mathcal{T}} \leftarrow \textsc{Normalize}(\mathcal{T}, \Sigma)$
$(\mathcal{T}^+, \mathcal{T}^-) \leftarrow \textsc{SplitByOutcome}(\widehat{\mathcal{T}})$
$S \leftarrow \textsc{MineSignatures}(\mathcal{T}^+)$
$C \leftarrow \textsc{BuildConnectors}(S, \widehat{\mathcal{T}})$
$\mathcal{P}, \mathcal{N} \leftarrow \emptyset, \emptyset$
for all $s \in S$ **do**
  $B \leftarrow \textsc{BuildSkeleton}(s, \Sigma, C)$
  $E \leftarrow \textsc{BindSupport}(s, \mathcal{T}^+)$
  $A \leftarrow \textsc{LLMCompile}(B, E)$
  $A \leftarrow \textsc{ValidateRepair}(A, B, E, \Sigma)$
  $\mathcal{P} \leftarrow \mathcal{P} \cup \{A\}$
end for
$\mathcal{N} \leftarrow \textsc{MineFailureGuards}(\mathcal{T}^-)$
return $(\mathcal{P}, \mathcal{N}, C)$

graph skeleton and its supporting traces. ValidateRepair checks these proposals against schemas and recorded evidence. It filters or demotes unsupported rules and can request limited revisions. Failed trajectories are processed separately to produce negative guards. This separates reusable structural evidence from model-proposed annotations. The distinction matters for execution ownership. The LLM proposes candidate rules offline, but neither a plausible rule nor a frequent historical sequence is sufficient to authorize a tool call at runtime.

Evidence-driven execution. Matching identifies a reusable fragment, but does not authorize its execution. Depending on the workload, the runtime retrieves candidates through semantic retrieval or state-aware structural matching. The Structure Engine then binds arguments from current evidence and checks each ready operation's guards and dependencies before invoking its tool. Tool results update the motif instance and may enable further operations. If no executable candidate is available, control returns to ReAct fallback.

Consider the refund task in Figure 1. Once an order is identified, the runtime can pass its identifier to the eligibility check through a compiled parameter-flow rule. No LLM call is needed for this transfer. If several orders remain plausible, the same execution instead requests a semantic choice. The distinction is not a fixed label on the tool step. It is whether the current evidence determines the argument.

Semantic handoff and resumption. A semantic gap should not force the agent to reconstruct work already completed. MotifAgent suspends the motif instance while retaining its execution position, tool observations, and validated bindings. A typed mediation request exposes only the unresolved slot, candidate, or guard and the evidence needed to resolve it. After mediation or fallback returns new evidence, the

runtime invalidates affected bindings, rechecks the pending transition, and resumes from the next unfinished operation.

Dynamic motif composition. Unlike fixed workflows, MotifAgent does not commit to an end-to-end execution path. As evidence evolves, the runtime can continue the current motif, connect to a compatible successor, or re-match another fragment. Successor entry bindings and action constraints are checked against the current state before execution. Different tasks can therefore share executable local structure while taking different paths. This avoids the choice between rigid end-to-end workflows and model-mediated decisions after every observation. MotifAgent can commit to a verified local transition without committing to the remainder of the task.

When tool schemas change or repeated failures expose outdated assumptions, affected motifs are revalidated or recompiled outside task-time execution.

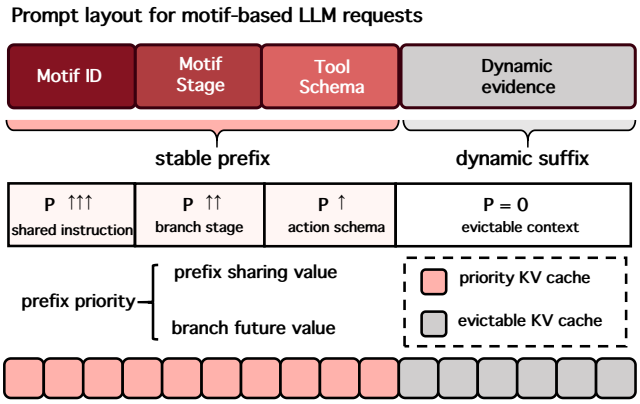


Figure 6. Fine-grained KV-cache retention prioritizes reusable motif prefixes across tool gaps while leaving dynamic evidence evictable.

4.3 Structure-Aware Scheduling

Prefix-aware schedulers exploit locality among requests already in the queue. They cannot directly see a request that an agent may release after a tool returns. Structure-Aware Scheduling (SAS) uses motif progress to expose this downstream opportunity without assuming that a predicted request will necessarily arrive. Crucially, this lookahead follows the evolving motif state rather than a static workflow DAG. A tool observation can change which successor is possible, so the backend treats future-prefix information as conditional.

Consider a waiting call A that may enable a later call B. Although B has not arrived, its predicted prefix may overlap with a queued request C. Executing A earlier could expose useful prefix sharing sooner. Yet doing so may delay other requests that already share cached prefixes. SAS balances potential downstream reuse against this immediate locality cost.

Serving hints. The runtime attaches `sglang_hints` to each remaining LLM call. These include motif and stage identifiers, a stable prefix ladder, and possible downstream calls inferred from the current motif stage and compiled connections to successor motifs. The ladder identifies nested stable prefix regions. Downstream prefixes inform scheduling, while prefix-range priorities and expiration times inform retention. The backend treats these hints as guidance rather than guaranteed future work.

Balancing reuse and locality. SAS estimates downstream reuse from the length of a predicted shared prefix and the number of queued requests that match it. It also estimates the cost of promoting a request ahead of a more densely shared radix-tree branch. For each branch along the promoted request's path, the scheduler compares queued-request counts with those of competing sibling subtrees. The largest positive difference provides a locality-cost estimate.

The estimated benefit, discounted by this cost, becomes a bounded bonus to DFS subtree scores. The existing backend then admits requests under its normal batch-size and memory limits. With speculative release credit disabled, a predicted downstream prefix receives credit only if it matches an actual queued request. Without an accepted positive bonus, SAS retains the base DFS ordering.

4.4 Fine-Grained KV-Cache Retention

Agent tool gaps create a cache-management problem that request-level recency alone cannot solve. A prefix may be needed after a tool returns, even if no current request uses it. Retaining an entire prompt can protect that prefix but also wastes space on dynamic evidence unlikely to recur. MotifAgent instead uses motif progress to prioritize selected reusable prefix ranges. The goal is not to pin as many tokens as possible. It is to retain context likely to serve future semantic calls while allowing task-specific observations to yield cache space to other requests.

Figure 6 separates stable motif context and tool schemas from task-specific inputs and observations. Motif stage and possible downstream calls indicate which stable prefix regions may be reused. Each prefix range receives a retention priority based on estimated reuse value per occupied KV block, with layer-dependent weights favoring upstream prefixes. The backend assigns these priorities and expiration times to radix-tree nodes, splitting nodes at range boundaries when needed. Cache hits still require exact token-prefix matches.

Under memory pressure, lower-priority ranges are evicted first. Higher-priority ranges remain reclaimable, and expired hints no longer add priority. This favors likely reuse across tool gaps without reserving cache indefinitely.

5 Implementation

MotifAgent is implemented as a Python agent runtime integrated with a modified SGLang v0.5.9 backend.

Agent layer. An offline compiler builds motif libraries from tool-use traces. During execution, the runtime retrieves motif candidates using workload-specific matching policies. Each selected motif maintains a `MotifFrame` containing its execution position, completed tools, tool results, unresolved parameters, and validated semantic bindings. Tool observations update frame bindings, release newly ready nodes, and expose possible successors through connector hints. Frames are preserved across suspension and fallback, while evidence dependencies are tracked to invalidate stale bindings before reuse. Typed semantic responses are resolved and validated before execution. Updated runtime evidence can also trigger motif re-selection or continuation through connector links. During ReAct fallback, the runtime matches tool sequences and error signatures against negative motifs to identify known failure patterns. Matches can trigger early termination or provide recovery guidance to the LLM.

System interface layer. The runtime serializes structural metadata into `sglang_hints`, carried through the OpenAI SDK's `extra_body` field. The backend parses these fields into `StructuredRequestHints` and propagates them with internal generation requests. The metadata includes motif and stage identifiers, prefix ladders, downstream-release estimates, cache-retention ranges, priorities, and expiration times. The backend uses these hints for scheduling and retention priorities, while KV-cache lookup remains based on exact token prefixes. Retention offsets are clamped to the cached token length and rounded down to cache-page boundaries.

Serving layer. The modified scheduler uses motif-stage and downstream-release hints to estimate future prefix reuse from queued requests. It applies bounded motif-aware adjustments to DFS subtree scores and reorders the waiting queue. Existing admission logic then selects requests under memory and batch-size constraints. The radix cache splits nodes at normalized retention boundaries and assigns priorities and expiration times to the corresponding nodes. Under cache pressure, eviction favors lower-priority KV blocks.

6 Evaluation

We evaluate two claims separately. First, making recurring tool-use structure executable should reduce model work while maintaining comparable task performance. Second, exposing motif progress should improve the serving of LLM calls that remain. Since runtime execution and serving can affect each other, we evaluate their contributions separately. We use agent benchmarks for the first claim and controlled request replays for the second.

6.1 Runtime-Layer Setup

Setup. The experimental host has one NVIDIA A100 GPU with 80 GB of memory and dual Intel Xeon Gold 6348 CPUs. Unless otherwise stated, we use DeepSeek-V4-Flash [6, 7] for runtime-layer LLM calls.

Workloads and metrics. We evaluate on APIBank [20], TauBench [51], and AppWorld [40], which cover single-turn tool use, multi-turn tool use, and API-based application automation. We use the official evaluators and report APIBank accuracy, TauBench task success rate, and AppWorld mean task pass percentage. Model work is measured by LLM calls and total prompt-plus-completion tokens per episode. We include final responses but exclude offline compilation throughout the runtime experiments.

Data splits. Compilation and evaluation use disjoint task IDs. We freeze the motif library before testing. APIBank uses three splits of its 50 Level-3 task IDs into compilation and evaluation pools. TauBench uses three splits, each with 58 compilation and 57 evaluation task IDs. AppWorld uses 90 training tasks for compilation and 100 `test_normal` tasks for evaluation. Each split or repetition contains 200 compilation inputs and 200 evaluation episodes, distributed as evenly as possible across task IDs.

Baselines. We compare MotifAgent with ReAct [52], ExpeL [58], AWM [44], Agent-KB [38], and APC [56]. ReAct provides the LLM-centric baseline. The other methods reuse experience or memory to guide execution. All methods run the same evaluation episodes within each split or repetition. Methods that require experience receive the same source trajectories from the compilation pool.

Motif matching. APIBank and TauBench use locally deployed `all-MiniLM-L6-v2` embeddings for initial retrieval, with cosine-similarity thresholds of 0.60 and 0.50, respectively. AppWorld first scores structural compatibility using tool schemas, task intent, entity cues, parameter readiness, and terminal actions, with a threshold of 0.90. Embedding retrieval provides a fallback. Runtime state and motif connections guide subsequent re-matching and continuation.

6.2 Runtime-Layer Results

The main result is consistent reduction in LLM work with comparable task performance. Table 1 shows that MotifAgent uses fewer tokens and LLM calls than every evaluated baseline on all three benchmarks. Relative to ReAct, APIBank accuracy rises from 56.3% to 76.8%, and TauBench task success rises from 62.5% to 74.0%. On AppWorld, MotifAgent reduces tokens by 36.3% and calls by 27.1%, while mean task pass percentage changes from 67.5% to 66.3%.

These savings come from two distinct changes: the runtime executes verified structural transitions without the model, and the remaining semantic calls use smaller contexts. This explains why token and call reductions need not match. On APIBank, for example, tokens fall by 47.9% while calls

Table 1. Runtime-layer results across three tool-use benchmarks. ↑ indicates higher is better and ↓ indicates lower is better. Best results are **bold** and second-best results are underlined.

Benchmark	Metric	ReAct	ExpeL	AWM	Agent-KB	APC	Motif (ours)
APIBank	Accuracy (%) ↑	56.33	72.00	67.67	67.00	**78.50**	76.83
	Avg. Tokens (k) ↓	9.63	16.09	11.68	11.53	6.42	**5.02**
	Avg. LLM Calls ↓	3.61	3.64	3.73	3.66	4.72	**3.38**
TauBench	Task success rate (%) ↑	62.50	74.00	**78.00**	72.00	69.00	74.00
	Avg. Tokens (k) ↓	94.91	100.67	104.87	98.93	90.49	**78.80**
	Avg. LLM Calls ↓	14.84	14.00	14.34	14.51	15.74	**11.97**
AppWorld	Pass percentage (%) ↑	67.48	**69.88**	62.81	67.87	56.37	66.26
	Avg. Tokens (k) ↓	204.17	213.67	205.77	245.84	193.37	**130.04**
	Avg. LLM Calls ↓	17.74	17.51	17.68	17.40	19.42	**12.93**

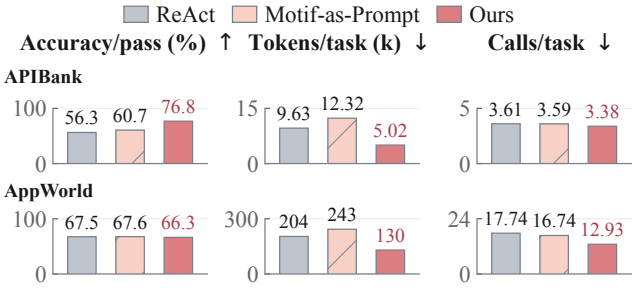


Figure 7. Executing motifs versus using them as ReAct prompt hints, with the same library and matcher.

fall by 6.4%. We separate these effects in §6.4. First, we test whether the same motif knowledge would suffice if it were only supplied as a prompt.

6.3 Motif as Prompt

Is the benefit due to reusable procedural knowledge, or to transferring execution control from the LLM to the runtime? We isolate this distinction with a *motif-as-prompt* variant. It uses the same compiled motif library and matcher as MotifAgent, but gives each matched motif to a ReAct agent as a prompt hint. The LLM still interprets the motif and performs every tool transition.

The prompt hint alone does not reproduce the runtime savings (Figure 7). On APIBank, motif-as-prompt raises accuracy from 56.3% to 60.7% but increases average tokens from 9.63k to 12.32k. With executable motifs, MotifAgent reaches 76.8% accuracy using 5.02k tokens and 3.38 LLM calls per episode.

On AppWorld, motif-as-prompt changes mean task pass percentage from 67.5% to 67.6% while increasing tokens from 204k to 243k. MotifAgent achieves 66.3% with 130k tokens and reduces calls from 17.74 to 12.93. The difference is architectural. Prompt hints still require the LLM to issue structurally determined actions, whereas executable motifs let the runtime perform argument binding and ready-node transitions directly.

6.4 Breakdown Experiments

Sources of runtime savings. Runtime savings arise in two ways: some model decisions disappear, while the semantic decisions that remain receive less context. Figure 8 measures these effects separately. The left panel attributes avoided structural LLM decisions to compiled-step dispatch, dataflow resolution, and iteration expansion. Each contribution is normalized over the avoided structural decisions within its benchmark. Iteration expansion accounts for 83.1% on TauBench. Dataflow resolution and iteration expansion contribute almost equally on APIBank, while compiled-step dispatch accounts for 63.1% on AppWorld.

The right panel measures input-token reduction for semantic operations within matched motifs, relative to ReAct-style calls. These operations use 74.9%, 87.5%, and 61.2% fewer input tokens on APIBank, TauBench, and AppWorld. They still invoke the LLM for semantic judgment, but supply motif state and relevant local evidence instead of the full execution history.

Task difficulty. Structural reuse is not limited to the easiest tasks. We group AppWorld episodes by the benchmark's Level 1–3 difficulty labels (Figure 9). Task pass percentage falls and model work grows with difficulty across methods, but MotifAgent uses fewer tokens and LLM calls at every level.

6.4.1 Model Robustness. Motif compilation and task-time semantic mediation rely on different model capabilities. To separate them, we vary the compiler and execution model independently using Qwen3.7-Plus (Plus) and Qwen3.5-9B (9B). Table 2(a) holds the Plus-compiled library fixed while

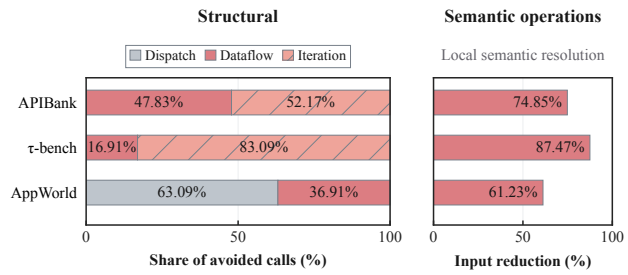


Figure 8. Runtime savings breakdown. Left: each mechanism's share of avoided structural LLM decisions. Right: input-token reduction for local semantic operations relative to ReAct-style calls.



Figure 9. AppWorld task performance and execution cost by benchmark-provided difficulty level.

changing execution models. Table 2(b) holds Plus execution fixed while changing compilers.

All configurations run the same 100 APIBank episodes, comprising four executions of each of 25 task IDs held out from compilation. Both compilers receive the same 200 training trajectories. ExpeL uses frozen knowledge derived from these training trajectories.

With the Plus-compiled library, MotifAgent reaches 72.0% accuracy under either execution model. Relative to ExpeL, it reduces tokens by 72.8–73.4% and calls by 15.8–16.3%. Accuracy exceeds ReAct with both models, matches ExpeL with 9B, and is lower than ExpeL with Plus.

Compilation quality matters more in this comparison. With Plus execution fixed, switching from the Plus compiler to 9B lowers accuracy from 72.0% to 64.0%, while token costs remain close at 4.43k and 4.27k. The runtime can reuse a library across execution models, but a weaker compiler may produce less effective motifs.

6.5 Serving-Layer Setup

Workloads. The replay experiments isolate scheduling and cache effects on requests already produced by the agent. Scheduling and retention use APIBank-derived LLM requests. AppWorld replays use two offered-load settings, $\rho = 0.3$ and $\rho = 2.1$, where $\rho = \lambda/\mu_{\text{ref}}$. Here, λ is the request arrival rate and $\mu_{\text{ref}} = C/s_{\text{ref}}$ is a reference capacity estimated from concurrency limit C and reference service time s_{ref}. All backend policies receive the same recorded prompts and target output

Table 2. Compilation and execution models on 100 APIBank episodes. Metrics follow §6.1.

(a) Execution models; compiled with Plus

Exec.	Method	Acc. (%)	Tok. (k)	Calls
Plus	ReAct	61.0	9.85	3.54
	ExpeL	76.0	16.68	3.68
	MotifAgent	72.0	4.43	3.08
9B	ReAct	64.0	9.58	3.48
	ExpeL	72.0	16.34	3.61
	MotifAgent	72.0	4.45	3.04

(b) Compilation models; executed with Plus

Compiler	Acc. (%)	Tok. (k)	Calls
Plus	72.0	4.43	3.08
9B	64.0	4.27	2.90

lengths. Fixed output lengths hold decode work constant, while latency is measured anew in each run.

Request release. The APIBank replay preserves task-level call dependencies instead of treating requests as independent. The first request becomes *eligible* after task arrival and its recorded non-LLM delay. Each later request becomes eligible only after its predecessor completes and its own delay elapses. Arrival schedules and recorded delays are identical across policies, but request eligibility depends on each policy's completion times.

Platform and load. The APIBank comparisons use Qwen3-14B on one A100 80 GB PCIe GPU with a modified SGLang backend. Scheduling runs allow 49,152 KV tokens and 16 concurrent requests, with retention disabled. Retention runs allow 6,000 KV tokens and 8 concurrent requests, using LPM scheduling. Each run starts a fresh server with an empty cache.

Within each experiment, policies and repetitions share a frozen trace and Poisson task-arrival schedule. The traces contain 200 tasks and 671 recorded requests. The task-arrival rates are 6 tasks/s for scheduling and 0.75 tasks/s for retention.

Baselines. We compare SAS with LPM, DFS-weight, and the PLAS policy of Agentix. LPM prioritizes requests by their longest reusable token prefix. DFS-weight favors densely shared branches of the prefix tree [59]. PLAS prioritizes calls from programs that have received less completed service [25]. These baselines compare motif-stage lookahead with current prefix locality and program-level attained service.

For retention, we compare LRU, a coarse request-level policy (Soft), and fine-grained retention (Layered). Soft combines reuse values into one priority for the request's entire static prefix. Layered assigns priorities to individual prefix

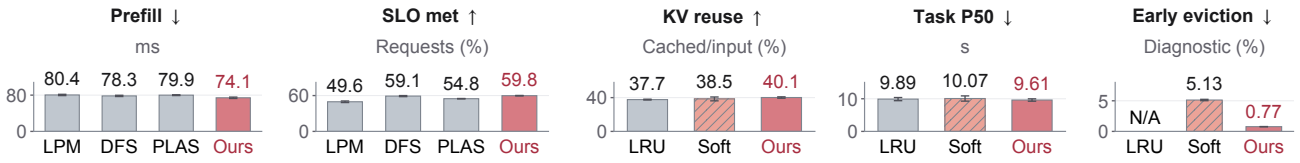


Figure 10. APIBank serving results: means over ten runs with 95% Student-t confidence intervals. Ours denotes SAS for scheduling and Layered for retention.

ranges. Both priority-based policies permit eviction under memory pressure.

Retention history. Soft and Layered use a binary recent-use signal to indicate whether a completed request used the same model and exact token prefix within the past 30 s. Retention priorities expire after 2 s.

Metrics. For scheduling, we report request-SLO attainment, mean prefill latency, and empirical latency distributions to assess deadline compliance and latency improvements. For retention, we report first-admission KV reuse, task P50 latency, and early-eviction rate to assess cache reuse, task latency, and premature eviction.

Request latency runs from eligibility to completion, including client submission delay, backend queueing, prefill, and decoding. Waiting for a predecessor and recorded non-LLM delays contribute to task latency rather than request latency. KV reuse is computed as the fraction of input tokens served from cache at first admission, while early eviction measures protected KV blocks evicted before their retention priorities expire.

SLO calibration. For APIBank, we set the request deadline to twice the request P95 measured in a serial, low-pressure run, yielding 5.72 s. The calibration run flushes the cache between tasks.

6.6 Serving-Layer Results

Scheduling. Downstream-aware ordering provides a modest gain over prefix-only scheduling (Figure 10). SAS achieves 74.1 ms mean prefill latency versus 78.3 ms for DFS-weight, with request-SLO attainment of 59.8% versus 59.1%. DFS-weight already exploits locality among queued requests. SAS also accounts for potential prefix-sharing successors, but its effect on request SLO is smaller because total latency includes submission delay, queueing, and decoding.

KV-cache retention. Selective retention performs better than assigning one priority to the whole static prefix. Soft raises mean first-admission KV reuse from LRU's 37.7% to 38.5%, but increases mean task P50 from 9.89 s to 10.07 s. Layered raises reuse to 40.1% while reducing task P50 to 9.61 s. Its early-eviction rate is 0.77%, versus 5.13% for Soft. Higher cache reuse alone does not ensure lower task latency. Layered protects selected reusable regions while keeping other ranges available for eviction.

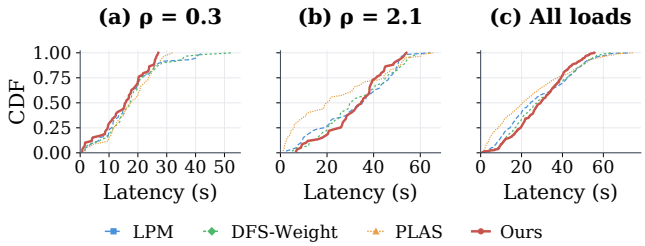


Figure 11. Per-request latency CDFs across scheduling policies in the AppWorld replay

AppWorld latency. The AppWorld replay shows that scheduling benefits depend on load. Figure 11 compares latency distributions under both load settings and in the combined replay. Structure-informed scheduling gives the clearest improvement in the tail, while policy ordering near the median varies across loads.

6.7 Overhead and Scalability Analysis

Compilation overhead. Compilation has an upfront cost, but a motif can serve many subsequent tasks. To examine this reuse, we feed successful AppWorld traces to the compiler in shuffled order. Figure 12 shows that the library stabilizes at 32 motif signatures after 54 candidates are submitted to the LLM-assisted compiler. The last 50 traces add no new compile candidate, indicating that later traces increasingly revisit covered structure.

Recorded executions suggest that a compilation event producing three new motifs uses 12.5k tokens. This is 6.12% of ReAct's mean per-task AppWorld token cost, or 2.04% per motif when divided across the three. Unlike task-time LLM calls, this compilation work need not be repeated whenever the same motif is reused.

Sensitivity to motif coverage. Serving lookahead depends on how many requests expose motif structure. We define a *motif request* as an LLM request built using motif-provided structural information. Coverage here is the fraction of these requests in the serving workload, not the fraction of tool calls skipped by the runtime. Requests without motif structure share the same queue.

We vary this fraction in a separate controlled APIBank dependency replay with successor releases and 100 requests

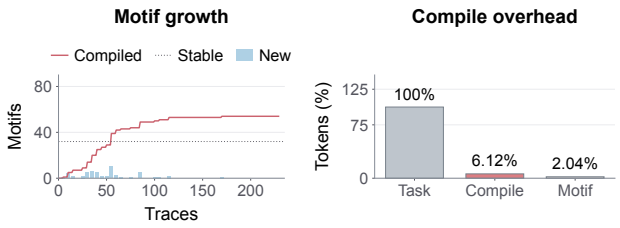


Figure 12. AppWorld motif-library growth and compilation cost, normalized to ReAct's mean per-task tokens.

Table 3. Task-P50 reduction relative to LPM (mean ± sample standard deviation over three paired seeds).

Coverage (%)	20	35	50	65
P50 reduction (%)	1.0 ± 2.5	4.4 ± 6.5	31.6 ± 3.0	42.6 ± 1.5

per policy. Table 3 reports task-P50 reduction relative to LPM over three paired seeds, using the arrival-to-completion definition in §6.5. Mean P50 reduction grows from 1.0% at 20% coverage to 42.6% at 65%. The result illustrates the potential value of lookahead when more queued work is connected to motif stages in this controlled setting.

6.8 Motif Behavior and Failure Analysis

Structural progress and task success are different outcomes. We group AppWorld episodes by positive-motif execution: a *guarded miss* has no executable positive motif, a *failed hit* executes one partially before falling back, and a *successful hit* completes motif execution. These labels do not indicate whether the full task passes the benchmark evaluator.

Even when no positive motif is executable, the system can retain fallback benefits. In the guarded-miss group, mean task pass percentage rises from ReAct's 44.4% to 52.8%, while tokens and calls fall by more than half (Figure 13). Negative motifs remain available to guide recovery or flag repeated failures.

Partial motif execution also preserves useful work. In the failed-hit group, MotifAgent achieves 69.4% mean task pass percentage versus 70.8% for ReAct, while reducing tokens from 206k to 154k. The runtime retains completed transitions across fallback rather than starting the fragment again.

Completing a motif gives the largest local cost reduction, but does not guarantee the task outcome. In the successful-hit group, tokens fall from 180k to 79k and calls from 16.7 to 9.7, while mean task pass percentage falls from 68.0% to 63.5%. A valid local fragment may still omit task-specific steps or global constraints, and semantic decisions outside the fragment remain consequential. Motif guards establish local validity, not end-to-end correctness.

Negative-guard selectivity. Negative guards aim to stop repeated failure patterns without treating every mismatch as

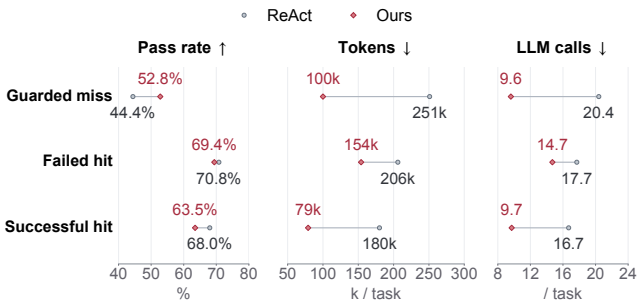


Figure 13. AppWorld task outcomes and execution costs for guarded misses, partial motif execution, and completed motif execution.

terminal. We test their selectivity on 168 recorded AppWorld ReAct trajectories in a five-fold shadow evaluation. Patterns are learned from task-disjoint training traces, and a repeated match is required before suggesting a stop. The shadow run records proposed stopping points but leaves the original continuations available for inspection.

The guards flag 20 continuations, all of which end in failure in the recorded trajectories. These represent 20 of 145 failures (13.8%). The saved continuations contain 333 LLM calls after the flagged points. Those calls would be avoided by stopping at the indicated points in these recorded executions.

7 Discussion and Limitations

7.1 Why This Is a Systems Problem

The motif-as-prompt ablation shows why this is a systems problem. Even with the same motif library and matcher, prompt hints leave every tool transition under LLM control. Reusable knowledge alone therefore cannot eliminate repeated model decisions. MotifAgent instead executes evidence-validated transitions in the runtime and preserves progress across semantic handoffs and fallbacks. This changes the ownership of execution control rather than merely optimizing individual model calls. This design also exposes potential structural information to the serving backend.

7.2 When MotifAgent Helps

MotifAgent is most effective when reusable tool-use patterns recur, tool schemas and parameter dependencies remain stable, and runtime evidence can verify transitions. We preserve LLM decision-making for steps requiring complex semantic reasoning or lacking sufficient runtime evidence.

Serving policies provide additional opportunities under queueing and KV-cache pressure, especially when motif progress helps predict downstream calls and reusable prefixes. These gains are expected to be smaller under light load or limited prefix reuse.

7.3 Limitations and Future Work

Motifs are compiled offline and fixed during evaluation. Future work could explore online motif discovery and recursive self-improvement, progressively converting repeated semantic decisions into evidence-verified structural transitions to investigate whether this process can approach the semantic critical-path bound $\mathrm{SCP}(\tau)$ discussed in Section 4.1.

We use task-disjoint splits within benchmarks, without testing cross-benchmark transfer or schema perturbations. A positive motif hit verifies local progress but does not establish global task success or optimality, while negative guards may prematurely terminate recoverable tasks.

Our design primarily targets tool-use structures. Code provides another powerful representation of executable structure, including control flow and data dependencies. Integrating code-level structures with motifs could extend structure–semantic separation to coding agents.

Isolated serving experiments show modest gains dependent on load, cache pressure, and motif coverage. Our runtime also changes prompt organization, potentially reducing cross-turn prefix reuse. When ReAct already benefits from substantial KV-cache reuse, fewer input tokens may not yield lower end-to-end latency. Furthermore, our analytical model suggests that decode dominates the remaining inference cost. Future work could explore structure-guided decoding that exploits structural information to accelerate generation without discarding information needed for correct execution.

8 Related Work

8.1 LLM Agents and Reusable Tool-Use Structure

Agent architectures use model-mediated action selection [31, 52], separate planning from observations [48], or execute generated code [28]. Experience-reuse systems retain insights, workflows, or plans for later tasks [38, 44, 56, 58]. When supplied as prompt guidance, this knowledge leaves the model to interpret the procedure and issue its calls. Programming abstractions and tool composition provide explicit ways to organize execution [4, 15, 24, 59].

Closer to our focus, AutoTool learns a tool inertial graph [13], AutoFlow synthesizes workflows [21], and GraSP executes verified skill graphs with local repair [47]. Other work explores graph reasoning, parallel calls, and workflow search [3, 16, 55]. MotifAgent reuses local fragments whose surrounding task paths may differ. It checks parameter flow and guards against current evidence, resolves semantic gaps through typed mediation, and re-matches or connects motifs as execution progresses.

8.2 LLM Serving and KV-Cache Management

Serving research improves batching and scheduling [2, 8, 32, 45, 53], KV allocation and prefix reuse [17, 34, 59], and request placement through migration or prefill–decode disaggregation [26, 35, 60]. These techniques optimize submitted requests. MotifAgent also avoids model calls for verified structural transitions.

Ayo exposes dataflow [36], and Parrot's semantic variables connect requests while preserving prompt structure [22]. Pie exposes serving-side handlers for programmable generation, KV management, and I/O [10]. Agentix schedules programs by attained service [25]. MotifAgent derives lookahead from evidence-validated motif progress. Observations can change which downstream calls become ready, so the backend combines this evolving state with the actual queue and cache.

KV-cache work addresses storage efficiency [27, 33], movement and selective retention [18, 23, 29, 57], and cross-prompt reuse [9, 50]. InferCept and Continuum address interrupted or multi-turn execution [1, 19]. MotifAgent uses motif stages to prioritize prefix ranges likely to recur after tool gaps. These priorities are temporary and permit eviction under pressure. The hints guide retention, while exact token-prefix matches determine reuse.

8.3 Workflow Scheduling

Dependency-aware schedulers exploit precedence and critical paths [11, 14], resource heterogeneity and changing load [39, 41], or workflow structure for data movement and task packing [5, 12]. Agent serving couples this lookahead with token-prefix locality. Promoting a predecessor may enable a useful successor, but observations can delay or change that successor. Promotion can also displace currently prefix-sharing requests, while retention consumes KV space. MotifAgent therefore weighs predicted downstream reuse against current locality and cache pressure.

9 Conclusion

MotifAgent addresses the LLM-centric architectural bottleneck by changing who controls routine execution. Executable motif graphs let the runtime advance evidence-supported tool transitions; typed mediation reserves the LLM for semantic gaps. The same motif state gives the serving backend advisory scheduling and cache hints. Across three benchmarks, MotifAgent reduces LLM calls and token consumption at comparable task performance, with serving replays showing complementary gains. The broader lesson is to make verifiable agent structure executable, rather than repeatedly asking the model to reconstruct it.

Acknowledgments

We thank the reviewers for their constructive comments. This work was supported by New Generation Artificial Intelligence-National Science and Technology Major Project (No. 2025ZD0122103), Fundamental and Interdisciplinary Disciplines Breakthrough Plan of the Ministry of Education of China (No. JYB2025XDXM118), Hubei Provincial Natural Science Foundation of China (No. 2026AFA002), and National Natural Science Foundation of China (No. 62621005).

References

[1] Reyna Abhyankar, Zijian He, Vikranth Srivatsa, Hao Zhang, and Yiying Zhang. 2024. InferCept: Efficient Intercept Support for Augmented Large Language Model Inference. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 81–95. https://proceedings.mlr.press/v235/abhyankar24a.html

[2] Amey Agrawal, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav S. Gulavani, and Ramachandran Ramjee. 2023. SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefills. *arXiv preprint arXiv:2308.16369* (2023).

[3] Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, and Torsten Hoefler. 2024. Graph of Thoughts: Solving Elaborate Problems with Large Language Models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 17682–17690. doi:10.1609/aaai.v38i16.29720

[4] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. 2023. Prompting Is Programming: A Query Language for Large Language Models. *Proceedings of the ACM on Programming Languages* 7, PLDI, Article 186 (2023), 24 pages. doi:10.1145/3591300

[5] Ewa Deelman, Karan Vahi, Gideon Juve, Mats Rynge, Scott Callaghan, Philip J. Maechling, Rajiv Mayani, Weiwei Chen, Rafael Ferreira da Silva, Miron Livny, and Kent Wenger. 2015. Pegasus, a Workflow Management System for Science Automation. *Future Generation Computer Systems* 46 (2015), 17–35. doi:10.1016/j.future.2014.10.008

[6] DeepSeek-AI. 2026. DeepSeek-V4-Flash (Model Card). Hugging Face. https://huggingface.co/deepseek-ai/DeepSeek-V4-Flash Accessed: 2026-10-16.

[7] DeepSeek-AI. 2026. DeepSeek-V4: Towards Highly Efficient Million-Token Context Intelligence. arXiv:2606.19348 [cs.CL] https://arxiv.org/abs/2606.19348

[8] Yichao Fu, Siqi Zhu, Runlong Su, Aurick Qiao, Ion Stoica, and Hao Zhang. 2024. Efficient LLM Scheduling by Learning to Rank. In *Proceedings of The Thirty-eighth Annual Conference on Neural Information Processing Systems*. https://doi.org/10.52202/079017-1882

[9] In Gim, Guojun Chen, Seung-seob Lee, Nikhil Sarda, Anurag Khandelwal, and Lin Zhong. 2024. Prompt Cache: Modular Attention Reuse for Low-Latency Inference. In *Proceedings of Machine Learning and Systems*, Vol. 6. 325–338.

[10] In Gim, Zhiyao Ma, Seung-seob Lee, and Lin Zhong. 2025. Pie: A Programmable Serving System for Emerging LLM Applications. In *Proceedings of the ACM SIGOPS 31st Symposium on Operating Systems Principles* (Lotte Hotel World, Seoul, Republic of Korea) *(SOSP '25)*. Association for Computing Machinery, New York, NY, USA, 415–430. doi:10.1145/3731569.3764814

[11] Ronald L. Graham. 1966. Bounds for Certain Multiprocessing Anomalies. *Bell System Technical Journal* 45, 9 (1966), 1563–1581.

[12] Robert Grandl, Srikanth Kandula, Sriram Rao, Aditya Akella, and Janardhan Kulkarni. 2016. GRAPHENE: Packing and Dependency-Aware Scheduling for Data-Parallel Clusters. In *Proceedings of 12th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, Savannah, GA, 81–97. https://www.usenix.org/conference/osdi16/technical-sessions/presentation/grandl_graphene

[13] Jingyi Jia and Qinbin Li. 2026. AutoTool: Efficient Tool Selection for Large Language Model Agents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 40. Association for the Advancement of Artificial Intelligence, Singapore, 31265–31273. doi:10.1609/aaai.v40i37.40389

[14] James E. Kelley and Morgan R. Walker. 1959. Critical-Path Planning and Scheduling. In *Proceedings of the Eastern Joint Computer Conference*. 160–173. doi:10.1145/1460299.1460318

[15] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan A, Saiful Haq, Ashutosh Sharma, Thomas Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling Declarative Language Model Calls into State-of-the-Art Pipelines. In *Proceedings of International Conference on Learning Representations*. 54928–54958. https://proceedings.iclr.cc/paper_files/paper/2024/hash/f1cf02ce09757f57c3b93c0db83181e0-Abstract-Conference.html

[16] Sehoon Kim, Suhong Moon, Ryan Tabrizi, Nicholas Lee, Michael W Mahoney, Kurt Keutzer, and Amir Gholami. 2024. An LLM Compiler for Parallel Function Calling. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. PMLR, 24370–24391. https://proceedings.mlr.press/v235/kim24y.html

[17] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.

[18] Wonbeom Lee, Jungi Lee, Junghwan Seo, and Jaewoong Sim. 2024. InfiniGen: efficient generative inference of large language models with dynamic KV cache management. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 9, 18 pages. https://dl.acm.org/doi/10.5555/3691938.3691947

[19] Hanchen Li, Runyuan He, Qiuyang Mang, Qizheng Zhang, Huanzhi Mao, Xiaokun Chen, Hangrui Zhou, Huanchen Zhang, Alvin Cheung, Joseph Gonzalez, and Ion Stoica. 2025. Continuum: Efficient and Robust Multi-Turn LLM Agent Scheduling with KV Cache Time-to-Live. *arXiv preprint arXiv:2511.02230* (2025). https://arxiv.org/abs/2511.02230v7 Version 7, revised September 8, 2026.

[20] Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 3102–3116.

[21] Zelong Li, Shuyuan Xu, Kai Mei, Wenyue Hua, Balaji Rama, Om Raheja, Hao Wang, He Zhu, and Yongfeng Zhang. 2024. Autoflow: Automated workflow generation for large language model agents. *arXiv preprint arXiv:2407.12821* (2024). https://doi.org/10.48550/arXiv.2407.12821

[22] Chaofan Lin, Zhenhua Han, Chengruidong Zhang, Yuqing Yang, Fan Yang, Chen Chen, and Lili Qiu. 2024. Parrot: Efficient Serving of LLM-based Applications with Semantic Variable. In *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*. USENIX Association, Santa Clara, CA, 929–945. https://www.usenix.org/conference/osdi24/presentation/lin-chaofan

[23] Yuhan Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, Michael Maire, Henry Hoffmann, Ari Holtzman, and Junchen Jiang. 2024. CacheGen: KV Cache Compression and Streaming for Fast Large Language Model Serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. Association for Computing Machinery, 38–56. doi:10.1145/3651890.3672274

[24] Pan Lu, Baolin Peng, Hao Cheng, Michel Galley, Kai-Wei Chang, Ying Nian Wu, Song-Chun Zhu, and Jianfeng Gao. 2023. Chameleon: Plug-and-Play Compositional Reasoning with Large Language Models. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 36)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/871ed095b734818cfba48db6aeb25a62-Abstract-Conference.html

[25] Michael Luo, Xiaoxiang Shi, Colin Cai, Tianjun Zhang, Justin Wong, Yichuan Wang, Chi Wang, Yanping Huang, Zhifeng Chen, Joseph E. Gonzalez, and Ion Stoica. 2026. Agentix: An Efficient Serving Engine for LLM Agents as General Programs. In *Proceedings of 23rd*

USENIX Symposium on Networked Systems Design and Implementation. USENIX Association, Renton, WA, 2443–2459. https://www.usenix.org/conference/nsdi26/presentation/luo

[26] Pratyush Patel, Esha Choukse, Chaojie Zhang, Aashaka Shah, Íñigo Goiri, Saeed Maleki, and Ricardo Bianchini. 2024. Splitwise: Efficient Generative LLM Inference Using Phase Splitting. In *Proceedings of the 51st ACM/IEEE International Symposium on Computer Architecture*. IEEE, 118–132. doi:10.1109/ISCA59077.2024.00019

[27] Ramya Prabhu, Ajay Nayak, Jayashree Mohan, Ramachandran Ramjee, and Ashish Panwar. 2025. vattention: Dynamic memory management for serving llms without pagedattention. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*. 1133–1150.

[28] Bo Qiao, Liqun Li, Xu Zhang, Shilin He, Yu Kang, Chaoyun Zhang, Fangkai Yang, Hang Dong, Jue Zhang, Lu Wang, Minghua Ma, Pu Zhao, Si Qin, Xiaoting Qin, Chao Du, Yong Xu, Qingwei Lin, Saravan Rajmohan, and Dongmei Zhang. 2023. TaskWeaver: A Code-First Agent Framework. *arXiv preprint arXiv:2311.17541* (2023). https://arxiv.org/abs/2311.17541

[29] Ruoyu Qin, Zheming Li, Weiran He, Jialei Cui, Heyi Tang, Feng Ren, Teng Ma, Shangming Cai, Yineng Zhang, Mingxing Zhang, et al. 2026. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *ACM Transactions on Storage* 22, 4 (2026), 1–38.

[30] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 36)*. Curran Associates, Inc., 68539–68551.

[31] Yongliang Shen, Kaitao Song, Xu Tan, Dongsheng Li, Weiming Lu, and Yueting Zhuang. 2023. HuggingGPT: Solving AI Tasks with ChatGPT and its Friends in Hugging Face. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 36)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/77c33e6a367922d003ff102ffb92b658-Abstract-Conference.html

[32] Ying Sheng, Shiyi Cao, Dacheng Li, Banghua Zhu, Zhuohan Li, Danyang Zhuo, Joseph E. Gonzalez, and Ion Stoica. 2024. Fairness in serving large language models. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 52, 24 pages. https://dl.acm.org/doi/10.5555/3691938.3691990

[33] Ying Sheng, Lianmin Zheng, Binhang Yuan, Zhuohan Li, Max Ryabinin, Beidi Chen, Percy Liang, Christopher Ré, Ion Stoica, and Ce Zhang. 2023. FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU. In *Proceedings of the 40th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 202)*. PMLR, 31094–31116. https://proceedings.mlr.press/v202/sheng23a.html

[34] Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. 2025. Preble: Efficient Distributed Prompt Scheduling for LLM Serving. In *Proceedings of International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/5bc342f48de8264779952fac378f96dc-Abstract-Conference.html

[35] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. 2024. Llumnix: dynamic scheduling for large language model serving. In *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation* (Santa Clara, CA, USA) *(OSDI'24)*. USENIX Association, USA, Article 10, 19 pages. https://dl.acm.org/doi/10.5555/3691938.3691948

[36] Xin Tan, Yimin Jiang, Yitao Yang, and Hong Xu. 2025. Towards End-to-End Optimization of LLM-based Applications with Ayo. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) *(ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 1302–1316. doi:10.1145/3676641.3716278

[37] Xiaoyu Tan, Bin Li, Xihe Qiu, Chao Qu, Wei Chu, Yinghui Xu, and Yuan Qi. 2025. Meta-agent-workflow: Streamlining tool usage in llms through workflow construction, retrieval, and refinement. In *Companion Proceedings of the ACM on Web Conference 2025*. 458–467.

[38] Xiangru Tang, Tianrui Qin, Tianhao Peng, Ziyang Zhou, Daniel Shao, Tingting Du, Xinming Wei, Peng Xia, Fang Wu, He Zhu, et al. 2025. Agent kb: Leveraging cross-domain experience for agentic problem solving. *arXiv preprint arXiv:2507.06229* (2025).

[39] Haluk Topcuoglu, Salim Hariri, and Min-You Wu. 2002. Performance-Effective and Low-Complexity Task Scheduling for Heterogeneous Computing. *IEEE Transactions on Parallel and Distributed Systems* 13, 3 (2002), 260–274.

[40] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. 2024. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, 16022–16076. doi:10.18653/v1/2024.acl-long.850

[41] Alexey Tumanov, Timothy Zhu, Jun Woo Park, Michael A. Kozuch, Mor Harchol-Balter, and Gregory R. Ganger. 2016. TetriSched: global rescheduling with adaptive plan-ahead in dynamic heterogeneous clusters. In *Proceedings of the Eleventh European Conference on Computer Systems*. Association for Computing Machinery, 35:1–35:16. doi:10.1145/2901318.2901355

[42] Noppanat Wadlom, Junyi Shen, and Yao Lu. 2026. Efficient LLM Serving for Agentic Workflows: A Data Systems Perspective. *Proceedings of the ACM on Management of Data* 4, 3, Article 169 (2026), 29 pages. doi:10.1145/3802046

[43] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, Wayne Xin Zhao, Zhewei Wei, and Ji-Rong Wen. 2024. A survey on large language model based autonomous agents. *Frontiers of Computer Science* 18, 6 (2024), 186345. https://doi.org/10.1007/s11704-024-40231-1

[44] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent Workflow Memory. In *Proceedings of the 42nd International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 267)*. PMLR, 63897–63911. https://proceedings.mlr.press/v267/wang25bx.html

[45] Bingyang Wu, Yinmin Zhong, Zili Zhang, Shengyu Liu, Fangyue Liu, Yuanhang Sun, Gang Huang, Xuanzhe Liu, and Xin Jin. 2026. FastServe: Iteration-Level Preemptive Scheduling for Large Language Model Inference. In *Proceedings of 23rd USENIX Symposium on Networked Systems Design and Implementation (NSDI 26)*. USENIX Association, 57–74. https://www.usenix.org/conference/nsdi26/presentation/wu-bingyang

[46] Zhiheng Xi, Wenxiang Chen, Xin Guo, Wei He, Yiwen Ding, Boyang Hong, Ming Zhang, Junzhe Wang, Senjie Jin, Enyu Zhou, Rui Zheng, Xiaoran Fan, Xiao Wang, Limao Xiong, Yuhao Zhou, Weiran Wang, Changhao Jiang, Yicheng Zou, Xiangyang Liu, Zhangyue Yin, Shihan Dou, Rongxiang Weng, Wenjuan Qin, Yongyan Zheng, Xipeng Qiu, Xuanjing Huang, Qi Zhang, and Tao Gui. 2025. The Rise and Potential of Large Language Model Based Agents: A Survey. *Science China Information Sciences* 68, 2 (2025), 121101. doi:10.1007/s11432-024-4222-0

[47] Tianle Xia, Lingxiang Hu, Yiding Sun, Ming Xu, Lan Xu, Siying Wang, Wei Xu, and Jie Jiang. 2026. GraSP: Graph-structured skill compositions for LLM agents. *arXiv preprint arXiv:2604.17870* (2026).

[48] Binfeng Xu, Zhiyuan Peng, Bowen Lei, Subhabrata Mukherjee, Yuchen Liu, and Dongkuan Xu. 2023. ReWOO: Decoupling Reasoning from Observations for Efficient Augmented Language Models. *arXiv preprint arXiv:2305.18323* (2023). https://arxiv.org/abs/2305.18323

[49] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. 2024. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 37)*. Curran Associates, Inc., 50528–50652. doi:10.52202/079017-1601

[50] Jiayi Yao, Hanchen Li, Yuhan Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. Association for Computing Machinery. doi:10.1145/3689031.3696098

[51] Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2025. τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains. In *Proceedings of International Conference on Learning Representations*. 9965–10017. https://proceedings.iclr.cc/paper_files/paper/2025/hash/1b126cc38b8638e07bef37e7b2bb72bf-Abstract-Conference.html

[52] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *Proceedings of International Conference on Learning Representations (ICLR)*.

[53] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. 2022. Orca: A Distributed Serving System for Transformer-Based Generative Models. In *Proceedings of 16th USENIX Symposium on Operating Systems Design and Implementation*. 521–538.

[54] Barry Zhang, Keith Lazuka, and Mahesh Murag. 2025. Equipping agents for the real world with Agent Skills. Anthropic Engineering. https://www.anthropic.com/engineering/equipping-agents-for-the-real-world-with-agent-skills Published October 16, 2025.

[55] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating Agentic Workflow Generation. In *Proceedings of International Conference on Learning Representations*. https://proceedings.iclr.cc/paper_files/paper/2025/hash/5492ecbce4439401798dcd2c90be94cd-Abstract-Conference.html

[56] Qizheng Zhang, Michael Wornow, and Kunle Olukotun. 2025. Agentic Plan Caching: Test-Time Memory for Fast and Cost-Efficient LLM Agents. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 38)*. Curran Associates, Inc., 114372–114398. doi:10.52202/085713-3451

[57] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. 2023. H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 36)*. https://proceedings.neurips.cc/paper_files/paper/2023/hash/6ceefa7b15572587b78ecfcebb2827f8-Abstract-Conference.html

[58] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. Expel: Llm agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 38. 19632–19642.

[59] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Sun, Jeff Huang, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2024. SGLang: Efficient Execution of Structured Language Model Programs. In *Proceedings of the Conference on Neural Information Processing Systems (Advances in Neural Information Processing Systems, Vol. 37)*. Curran Associates, Inc., 62557–62583. doi:10.52202/079017-2000

[60] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. 2024. DistServe: Disaggregating Prefill and Decoding for Goodput-optimized Large Language Model Serving. In *Proceedings of 18th USENIX Symposium on Operating Systems Design and Implementation*. USENIX Association, 193–210. https://www.usenix.org/conference/osdi24/presentation/zhong-yinmin